HIGH PERFORMANCE COMPUTING

ASSIGNMENT-10

Name : GINNE SAI TEJA
HT No : 2303A51526
Batch No : 22

Assignment 1: Non-Blocking Send and Receive with Overlap

Objective: Implement non-blocking point-to-point communication using Isend and Irecv and demonstrate overlap of computation with communication.

Python Code (assignment1.py)

```
from mpi4py import MPI
import numpy as np
import time

def compute_task(rank):
    """Simulates independent computation."""
    print(f"Rank {rank} is performing background computation...")
time.sleep(0.5) # Simulate workload
    print(f"Rank {rank} finished computation.")

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:     # Master process
data = np.arange(10, dtype='i')
print(f"Rank 0 sending data: {data}")
    req = comm.Isend(data, dest=1, tag=11)

    # Overlap computation with communication
compute_task(rank)

    req.Wait()
    print("Rank 0: Send completed.")

elif rank == 1:     #
Worker process
data =
```

```
np.empty(10,
dtype='i')
   req = comm.Irecv(data, source=0, tag=11)

   # Overlap computation with communication
compute_task(rank)

   req.Wait()
   print(f"Rank 1: Receive completed. Data received: {data}")
```

Expected Output (mpiexec -n 2 python assignment1.py)

```
Rank 0 sending data: [0 1 2 3 4 5 6 7 8 9]
Rank 0 is performing background computation...
Rank 1 is performing background computation...
Rank 0 finished computation.
Rank 0: Send completed.
Rank 1 finished computation.
Rank 1: Receive completed. Data received: [0 1 2 3 4 5 6 7 8 9]
```

Assignment 2: Overlapping Neighbor Communication with Computation

Objective: Study non-blocking communication between neighboring processes and overlap it with computation.

Python Code (assignment2.py)

```
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Determine neighbors (wrapping around)
left = (rank - 1) % size
right = (rank + 1) % size

# Local data to send
```

```
send_data = np.array([rank * 10], dtype='i')
recv_data = np.empty(1, dtype='i')

print(f"Rank {rank} starting communication. Sending {send_data[0]} to {right}, receiving from {left}.")

# Non-blocking send to right, receive from left
req_s = comm.Isend(send_data, dest=right,
tag=22) req_r = comm.Irecv(recv_data, source=left,
tag=22)

# Computation during communication (Sum of local data)
local_sum = sum(range(1000000))

# Synchronize
MPI.Request.Waitall([req_s, req_r])

print(f"Rank {rank} completed. Received {recv_data[0]} from {left}. Background computation result:
{local_sum}")
```

Expected Output (mpiexec -n 3 python assignment2.py)

```
Rank 0 starting communication. Sending 0 to 1, receiving from 2.
Rank 1 starting communication. Sending 10 to 2, receiving from 0.
Rank 2 starting communication. Sending 20 to 0, receiving from 1.
Rank 0 completed. Received 20 from 2. Background computation result: 499999500000
Rank 1 completed. Received 0 from 0. Background computation result: 499999500000
Rank 2 completed. Received 10 from 1. Background computation result: 499999500000
```

Assignment 3: Non-Blocking Receive Using Test

Objective: Use the Test operation in MPI for monitoring non-blocking communication completion.

Python Code (assignment3.py)

```
from mpi4py import MPI
import numpy as np
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
```

```
if rank == 0:     data =
np.array([99], dtype='i')     #
Simulate a delayed send
time.sleep(1.5)
comm.Send(data, dest=1,
tag=33)
   print("Rank 0: Data sent.")

elif rank == 1:     data =
np.empty(1, dtype='i')
   req = comm.Irecv(data, source=0, tag=33)

   flag = False
   loops = 0

   print("Rank 1: Starting computation loop while waiting for data...")
while not flag:
      # Perform useful computation while waiting
      time.sleep(0.2)
loops += 1
      print(f"Rank 1: Doing work... (Cycle {loops})")

      # Periodically check communication status
flag = req.Test()

   print(f"Rank 1: Communication complete! Processing received data: {data[0]}")
print(f"Rank 1: Total computation cycles completed: {loops}")
```

Expected Output (mpiexec -n 2 python assignment3.py)

```
Rank 1: Starting computation loop while waiting for data...
Rank 1: Doing work... (Cycle 1)
Rank 1: Doing work... (Cycle 2)
Rank 1: Doing work... (Cycle 3)
Rank 1: Doing work... (Cycle 4)
Rank 1: Doing work... (Cycle 5)
Rank 1: Doing work... (Cycle 6)
Rank 1: Doing work... (Cycle 7) Rank
0: Data sent.
Rank 1: Doing work... (Cycle 8)
Rank 1: Communication complete! Processing received data: 99
```

Rank 1: Total computation cycles completed: 8

Assignment 4: Non-Blocking Ring Communication with Overlap

Objective: Implement ring communication using non-blocking MPI calls and overlap computation with data exchange.

Python Code (assignment4.py)

```
from mpi4py import MPI
import numpy as np
import time

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Ring topology logical arrangement
next_rank = (rank + 1) % size
prev_rank = (rank - 1) % size

send_data = np.array([rank], dtype='i')
recv_data = np.empty(1, dtype='i')

# Non-blocking exchange req_s = comm.Isend(send_data,
dest=next_rank, tag=44) req_r = comm.Irecv(recv_data,
source=prev_rank, tag=44)

# Overlap computation
start_time = time.time()
computation_result = sum(i * i for i in range(1000000))
calc_time = time.time() - start_time

# Wait for communication to finish
MPI.Request.Waitall([req_s, req_r])

print(f"Rank {rank}: Received data {recv_data[0]} from Rank {prev_rank}. "
f"Computed result {computation_result} in {calc_time:.4f} seconds.")
```

Expected Output (mpiexec -n 4 python assignment4.py)

Rank 0: Received data 3 from Rank 3. Computed result 333332833333500000 in 0.0521 seconds.
Rank 1: Received data 0 from Rank 0. Computed result 333332833333500000 in 0.0519 seconds.
Rank 2: Received data 1 from Rank 1. Computed result 333332833333500000 in 0.0522 seconds. Rank 3: Received data 2 from Rank 2. Computed result 333332833333500000 in 0.0518 seconds.

Assignment 5: Blocking vs Non-Blocking Communication Comparison

Objective: Compare blocking (Send/Recv) and non-blocking (Isend/Irecv) MPI communication and analyze computation overlap.

Python Code (assignment5.py)

```
from mpi4py import MPI
import numpy as np
import time

def workload():
    """Simulates a heavy computational workload (approx 0.5s)."""
time.sleep(0.5)

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

# Large payload to make communication time noticeable
payload_size = 5000000

if rank == 0:
    send_data = np.ones(payload_size, dtype='d')
elif rank == 1:
    recv_data = np.empty(payload_size, dtype='d')

comm.Barrier() # Synchronize before starting

# ========================================
# TEST 1: Blocking Communication
# ========================================
start_blocking = time.time()
```

```
if rank == 0:
   comm.Send(send_data, dest=1, tag=1)
   workload() # Computation is forced to wait until Send is done
elif rank == 1:
   comm.Recv(recv_data, source=0, tag=1)
workload() # Computation waits for Recv to finish

comm.Barrier()
end_blocking = time.time()
blocking_time = end_blocking - start_blocking

# ========================================
# TEST 2: Non-Blocking Communication
# ========================================
start_non_blocking = time.time()

if rank == 0:
   req = comm.Isend(send_data, dest=1, tag=2)
workload() # Computation overlaps with data transfer
req.Wait() elif rank == 1:
   req = comm.Irecv(recv_data, source=0, tag=2)
workload() # Computation overlaps with data transfer
req.Wait()

comm.Barrier()
end_non_blocking = time.time()
non_blocking_time = end_non_blocking - start_non_blocking

# ========================================
# Results
# ======================================== if
rank == 0:    print(f"Data Payload Size: {payload_size}
doubles")    print("-" * 40)
   print(f"Blocking Time Execution    : {blocking_time:.4f} seconds")
print(f"Non-Blocking Time Execution : {non_blocking_time:.4f} seconds")
print("-" * 40)    if non_blocking_time < blocking_time:
      print(f"Non-blocking was {blocking_time - non_blocking_time:.4f} seconds faster due to overlap.")
```

Expected Output (mpiexec -n 2 python assignment5.py)

Data Payload Size: 5000000 doubles

Blocking Time Execution : 0.5231 seconds
Non-Blocking Time Execution : 0.5015 seconds

Non-blocking was 0.0216 seconds faster due to overlap.